

Jour 3 — Automates d'arbres (pivot) + hash-consing

Module : `formlang/tree.py` • Apps : `morpho`, `shield`, `hashcons` • Barème : 5 pts (+2)

Cadre éthique

Volet Shield 100 % structurel : jetons abstraits, aucune attaque réelle.

Contexte

C'est le **pivot** du projet. Une séquence plate `p p r s` ne dit pas *comment* le mot est construit : la morphologie est **hiérarchique**, donc un **arbre**. On généralise l'automate fini : au lieu de suivre une *chaîne* d'états, on **fusionne** les états des enfants d'un nœud. Deux applications très différentes (morphologie, pare-feu structurel) et le hash-consinginstancient le **même** `formlang.tree`.

Notions nouvelles du jour

1. Alphabet classé, termes, et automate d'arbres ascendant (BUTA)

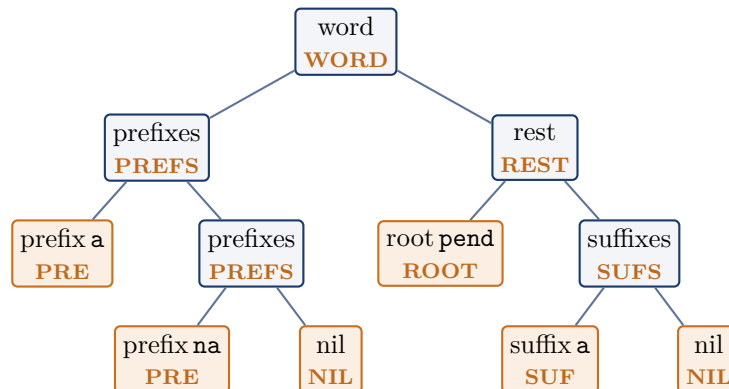
Chaque symbole a une **arité**. Un **BUTA** est $A = (Q, \Sigma, Q_f, \Delta)$ avec des règles $f(q_1, \dots, q_n) \rightarrow q$: « si f a ses enfants dans les états $q_1 \dots q_n$, ce nœud reçoit q ». On accepte si la **racine** reçoit un état de Q_f .

L'AFD généralisé

Un AFD lit un mot de gauche à droite : c'est le cas « tout en arité 1 » (chaîne). Le BUTA part des **feuilles** et **remonte** : à un nœud, il *combine* les états déjà calculés des enfants. • Code : `TreeAutomaton.run` (post-ordre ; REJECT si un enfant rejette ou si aucune règle).

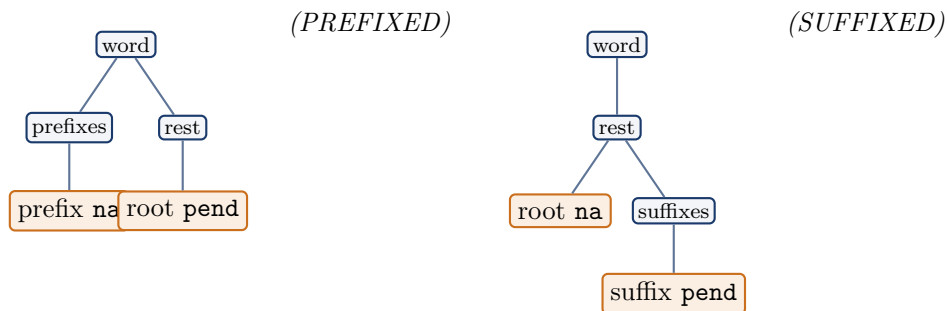
2. Exécution ascendante — exemple morphologique

Le mot circonfixé `a-na-pend-a` : l'état de chaque nœud (en **orange**) est calculé de bas en haut ; la racine reçoit `WORD` $\in Q_f \Rightarrow$ **accepté** (classe `CIRCUMFIXED`).



3. Pourquoi un AFD *de mots* échoue : la frontière ne suffit pas

La **frontière** (yield, la suite des feuilles) ne détermine pas la structure. Les deux arbres ci-dessous ont la **même** frontière **na pend** mais des structures — et des classes — *différentes*. Un AFD ne lit que la frontière : il est **aveugle** à cette différence.

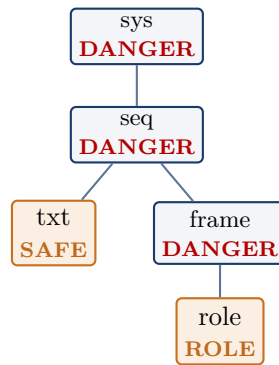


Théorème du yield — le pont vers le jour 1

La frontière d'un mot morphologique accepté est exactement $p^* r s^*$, le **langage régulier** du jour 1. Réciproquement, l'ensemble des frontières d'un automate d'arbres régulier est **hors-contexte**. Pont $arbres \leftrightarrow mots$. Déterminisme $\Rightarrow$ exécution unique et coût $O(n)$.

4. Le même moteur pour le Shield (100 % structurel)

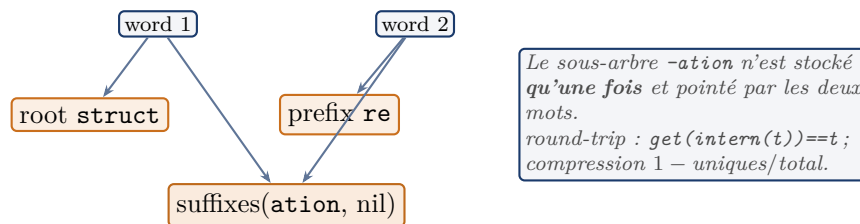
`shield_automaton` instancie le *même* BUTA sur des jetons abstraits. Exemple : `sys(seq(txt, frame(role)))` remonte à **DANGER** $\in Q_f$ — donc **bloqué**. Aucun contenu réel n'intervient, seulement la *forme*.



Règles clés : $\text{frame}(x) \rightarrow \text{SAFE}$ si $x = \text{SAFE}$ sinon DANGER ; seq propage DANGER ; d'où la remontée $\text{ROLE} \rightarrow \text{DANGER} \rightarrow \text{DANGER}$.

5. Clôture par intersection (produit) et hash-consing (DAG)

Les langages d'arbres réguliers sont clos par **intersection** : l'automate produit $A_1 \times A_2$ a pour états des *paires*, règles composante par composante (`formlang.tree.product`). Le **hash-consing** donne à chaque sous-arbre distinct un **id unique** : les sous-arbres identiques sont **partagés**, l'arbre devient un **DAG**.



Plus une langue est **agglutinante** (turc, finnois), plus le partage suffixal est élevé (pendant « arbres » des dictionnaires minimaux, Daciuk *et al.* 2000).

A. Théorie (à rédiger) — 2 pts

Q3.1 (0,5)

Montrer le déterminisme de `morpho` et `shield`; conséquence (unicité, $O(n)$).

Q3.2 (0,5)

Pourquoi un AFD de mots échoue ? Deux arbres de même frontière.

Q3.3 (0,5)

Théorème du yield : frontière p^*rs^* et lien au jour 1.

Q3.4 (0,5)

Réduplication : langage d'arbres régulier ? Intuition via $\{ww\}$. (Indice réel : Culy 1985 — le bambara sort du hors-contexte.)

B. Pratique — 3 pts

E3.1 (1)

`TreeAutomaton.run/accepts`. Débloque *test_tree.py*.

E3.2 (0,5)

`morpho_automaton` (règles Δ). Tests *morpho*.

E3.3 (0,5)

`_seq + shield_automaton`. Test *test_shield_blocage*.

E3.4 (0,5)

`product` : $L = L(A_1) \cap L(A_2)$. Test *test_produit_intersection*.

E3.5 (0,5)

`CompactStore.intern/get` (hash-consing). Tests *test_hashcons*.

Q3.5 (rapport) : compression = $1 - \text{uniques}/\text{total}$; plus élevée pour les langues agglutinantes. Reportez vos **vrais** chiffres.

Mini-barème

Item	Détail	Pts
Q3.1–Q3.4	déterminisme, yield, reduplication	2
E3.1	<code>run/accepts</code> du BUTA	1
E3.2	automate morphologique + <code>classify</code>	0,5
E3.3	<code>AttackDecomposer</code>	0,5
E3.4	produit (intersection)	0,5
E3.5	hash-consing (intern + round-trip)	0,5
Bonus	infixation <code>s<um>ulat</code> (étendre Δ)	+1
Bonus	minimisation d'un BUTA	+1

Gate (règle non négociable)

Les apps **instancient** `formlang.tree.TreeAutomaton` / Term. Toute réimplémentation d'un automate d'arbres perd les points d'intégration.

Références

Comon *et al.*, TATA (2007, libre). • Gécseg & Steinby (1984; rééd. 2015). • Thatcher & Wright 1968; Doner 1970 (*fondeurs*). • Harris 1955, Goldsmith 2001; Culy 1985. • Filliâtre & Conchon 2006.